PyMySQL

This document contains useful information on how to integrate Python and SQL using pymysql.

Note: The terminology included in this document and a basic understanding of Python SQL integration is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction

Due to the growing importance of relational database management, it is important to be able to integrate Python usage to SQL. One common and popular solution to this is PyMySQL. PyMySQL is a pure-Python MySQL client library that allows you to connect to a MySQL database server from Python. By using PyMySQL, users can run queries, read and write information, and retrieve/manipulate data all with Python.

Terminology

connection - a **connection** is a setup of resources so a particular object such as a file or a database can be read or written into.

cursor - a **cursor** is a pointer object that stores a memory address of the position where the current reference point is at.

transaction - a **transaction** is any sort of operation performed on a SQL database that modifies the state of the MySQL database.

commit - in SQL, a **commit** statement ends the current transaction within a SQL database and saves the changes made.

terminate - ending a program or process; stopping that particular session.

host - a **host** is any device/IP address where the database server is located.

fetch - in computer terminology, **fetch** is getting, reading, or moving data objects.

execute - **execute** is the process of running a command, script, or a program.

close - in computer terminology, to **close** is to terminate a program or exit a file.

Provided Files

This handout uses the following files for demonstrating examples:

- recitation_tas.sql
  - SQL script used to create the database and tables shown in this handout
- pymysql_dictcursor.py
  - Python script that uses **cursors.DictCursor** to connect to MySQL
- pymysql_cursor.py
  - Python script that uses **cursors.Cursor** to connect to MySQL

Python SQL Communication

Before starting on how to connect Python to MySQL with PyMySQL, you must first make sure the PyMySQL library is installed. Within your Terminal / Command Line prompt, run the following command:

```
>>> conda install pymysql
```

Follow the instructions that appear on your Terminal / Command Line prompt and verify that you have installed the PyMySQL library successfully. If you run into any errors, attend office hours to get your issue resolved immediately.

After a successful installation, you can now import the PyMySQL module and start connecting to MySQL with Python. We will now go over a basic structure of what a Python file using the PyMySQL library should look like.

For the following examples, we will be using the `CS2316` database with the following tables and data:

Table `roster`:

ID_number	name	major	ta_ID
1	Manny Jonson	ISYE	0
2	Xinran Yu	ISYE	1
3	Eric Yan	CS	3
4	Colleen Cabugason	ISYE	2
5	Marylyn Chen	ISYE	2
6	Josh Hall	ISYE	1
7	Bohong Cheng	ISYE	3

7 rows in set (0.00 sec)

Table `recitation`:

ta_ID	section	num_students	room
0	Helpdesk	0	CoC 273B
1	A02, A03, A05	48	Mason 2117
2	A04	46	IC 215
3	A01	52	Bunger-Henry 380

4 rows in set (0.00 sec)

Establishing a Connection

Let's first start by creating and opening a new .py file. In this handout, we will be using both the `pymysql_dictcursor.py` and `pymysql_cursor.py` files as examples. After creating your .py file, you must make sure to import the PyMySQL module or you will run into errors. Next, we will begin by establishing a connection with the MySQL server and database.

pymysql.connect()

To establish a connection to MySQL, we first need to construct a connection object. In order to do this, we can use the `.connect()` function from `pymysql`, which we have imported as a module. You can **return a connection object** by following the structure of code shown below:

```
import pymysql
connection = pymysql.connect(host = 'localhost', user = 'root',
          password = '__________', db = '__________', charset =
          'utf8mb4', cursorclass = pymysql.cursors.__________)
```

where the empty spaces denote your MySQL password, the name of the database you want to access, and the cursorclass you want to use[1]. This creates a connection object named `connection` and successfully establishes a connection between Python and your MySQL database. Although there are various charsets available for us to use, this course will focus on using '`utf8mb4`' for the charset.

Just like charsets, there are many types of cursor classes available to use. Two examples are the **Cursor** and **DictCursor** classes. The **Cursor** class takes the rows in databases and converts them into tuples, whereas the **DictCursor** class converts them to dictionaries for us to use. Here is what we have so far[2]:

```
>>> import sys, pymysql, getpass
>>> def main(password, db):
        connection = pymysql.connect(host='localhost', user='root',
                                     password=password,db=db,
                                     charset='utf8mb4',
                                     cursorclass=pymysql.cursors.DictCursor)
>>> if __name__ == '__main__':
        user_password = getpass.getpass("Enter your MySQL password")
        main(user_password, sys.argv[1])
```

[1] Note that the host name will usually be '`localhost`' or some IP address, the user will usually be '`root`', and charset will be '`utf8mb4`' in most cases. You can either choose the **Cursor** or **DictCursor** cursorclass. Using `sys.argv` argument variables are not required but encouraged here!

[2] If you **DO NOT** have a password for your MySQL server, you should set `password = ''` (empty string)

getpass Module

For security reasons, you should avoid hard coding your password into the source code file. In the previous section, we used `getpass.getpass("Enter your MySQL password: ")`. This `getpass.getpass` function can be used to read in a user's password, and acts just like the built-in `input` function. The main difference is that whatever the user enters is hidden, similar to a normal password field.

If you do not have a password, you can just press enter when prompted for your password, and an empty string will be used for the `user_password` variable.

Creating a Cursor

After creating a connection object, we now need to create a cursor object to help access the part of the database that we are interested in. When using the PyMySQL module, we can use the method `.cursor()` to help us define a new cursor object. We can do this by calling the `.cursor()` method onto our connection object that we defined in the previous section.

connection.cursor()

There are 2 ways you can use to create a cursor object:

Method 1:

```
cursor = connection.cursor()
```

Method 2:

```
with connection.cursor() as cursor:
```

In the example structure above, we have now **returned a cursor object** named `cursor` that will help us navigate through the parts of the database that we wish to access. Using the cursor object, you will be able to call methods on it to run queries and to extract information through Python.
Note: If you decide to construct a cursor using the `with connection.cursor() as cursor` line, the temporary cursor object you have defined will only exist within the `with` statement indentation. Breaking out of the indentation will **terminate** your cursor. If you leave the indentation and try to call methods on your temporary cursor, your code will error since the cursor object was terminated.

Communicating with the Database

Once you have established a cursor object, you can use different methods to interact with the MySQL database. In order to run query statements, we will need to use the `.execute()` and `.executemany()` methods to help us run the desired statements.

cursor.execute()

To run query statements within Python, we can call our cursor object's `.execute()` method. This method takes in a query, which is the statement you are trying to execute. **Make sure the query you want to run is a string!** When writing queries in PyMySQL, you can omit the semicolon at the end. The general format of the method is as follows:

Usage:

```
query = 'Query goes here'
cursor.execute(query)
```

The `.execute()` method **returns an int** of the number of rows affected after executing the query.

Note: For each query you want to execute, you must call the `.execute()` method for each query statement.

Example of using `cursor.execute` with a query:

```
>>> query = 'INSERT INTO roster VALUES (0, "Melinda McDaniel", "CS", 0)'
>>> cursor.execute(query)
```

The expression `cursor.execute(query)` would **return the int 1**, as 1 row is affected by the insert statement.

In addition to the method `.execute()`, we can use the method `.executemany()` to run multiple queries at once.

cursor.executemany()

At times, if you have a list of tuples that contain data for multiple queries you want to execute, you can use the `.executemany()` method. The `.executemany()` method is mainly used for insert statements.

Usage:

```
cursor.executemany('Query goes here', list_of_tuples)
```

The `.executemany()` method **returns an int** of the number of rows affected after executing the query many times using the data in `list_of_tuples`.

Here are two different ways of inserting data into a database table:

1) Using `cursor.execute()` in a loop instead of using `cursor.executemany()`:

```
>>> data = [(8, "Dwight Schrute", "Marketing", 0),
            (9, "Jim Halpert", "Marketing", 0)]
```

```
>>> for tup in data:
        cursor.execute("INSERT INTO roster VALUES (%s, %s, %s, %s)", tup)
```

2) Using `cursor.executemany()`:

```
>>> data = [(8, "Dwight Schrute", "Marketing", 0),
            (9, "Jim Halpert", "Marketing", 0)]
>>> cursor.executemany("INSERT INTO roster VALUES (%s, %s, %s, %s)", data)
```

The %s Character

You may encounter the `%s` character when reading SQL query statements written with PyMySQL. In PyMySQL, the `%s` character acts like a "wildcard" or "replacement variable"; if you remember f-string formatting, the `%s` is similar to `{}`. For example, the following two lines of code execute the same query:

```
cursor.execute('INSERT INTO roster VALUES (10, "Michael Scott", "Not Business", 0);')
```

```
cursor.execute('INSERT INTO roster VALUES (%s, %s, %s, %s);', (10, "Michael Scott", "Not Business", 0))
```

Notice that the "%s" characters in the query are replaced by the values in the tuple sequentially.

When running queries via PyMySQL, you should avoid using f-strings or string concatenation when integrating arbitrary values into your query string. Using f-strings or string concatenation leave your code vulnerable to SQL injection attacks on your databases. These vulnerabilities are one of the most commonly seen, and they can easily be prevented by using this `%s` placeholder, as PyMySQL will take care of safely building and executing the query.

Remember the second argument to execute should be a tuple, so if you want to use just one `%s` placeholder, you would pass in a tuple of one element as the second argument:

```
>>> cursor.execute("SELECT * from pages WHERE views >= %s",
                   (min_views,))
```

Extracting Data from a Resulted Query

Oftentimes we are interested in the result of the queries executed. In order for us to extract and use the resulting data from each query in Python, we will use the `.fetchone()` and `.fetchall()` methods.

cursor.fetchone()

This method **returns a dictionary** (if using pymysql.cursors.**DictCursor**) or **a tuple** (if using pymysql.cursors.**Cursor**) with all the data contained on a **single row** returned from the previous executed query. The cursor starts on the **first row** and each time this method is called, the cursor

scrolls to the next row until all rows are returned. Calling `.execute()` resets the cursor to the top row.

Usage:

```
cursor.fetchone()
```

cursor.fetchall()

This method **returns a list of dictionaries** (if using **DictCursor** cursorclass) or **a list of tuples** (if using **Cursor** cursorclass) with all the data contained on **all rows** returned from the previous executed query. The cursor scrolls through all the rows that are returned, thus the second time you call this method it will either **return an empty list** (if using **DictCursor** cursorclass) or **return an empty tuple** (if using Cursor cursorclass).

Usage:

```
cursor.fetchall()
```

Closing the Cursor and Connection

Once you are finished with executing your queries, you must commit and close both the cursor and connection object. This allows any changes made to the modified MySQL database to be saved. **Failure to commit the connection will result in database modifications to be unsaved.** It is in your best interest to make committing and closing your connection after your last executed query a habit.

cursor.close()

This method closes the cursor, resets all results, and ensures that the cursor object has no reference to its original connection object. Although closing the cursor is not always a necessary thing to do, this can help avoid future errors in case you are working with multiple cursors at once. Note that if you are using `with connection.cursor() as cursor`, the cursor is automatically terminated when you break out of the indention.

Usage:

```
cursor.close()
```

connection.commit()

Calling this method sends a commit statement to the MySQL server, committing/ending the current transaction. If you are modifying the database using INSERT, UPDATE, or DELETE statements it is important and conventional to call `connection.commit()` to finalize the transaction and save the changes that were made.

Usage:

```
connection.commit()
```

connection.close()

Calling this method on a connection object will terminate the current connection with the database on MySQL.

Usage:

```
connection.close()
```

PyMySQL Object Methods

Method	Description
pymysql.connect()	Connects to your MySQL database and **returns a connection object**
connection.cursor()	When called on a connection object, **returns a cursor object** that can run query statements when methods are called on it
cursor.execute()	Executes/runs the given SQL command/query onto the database of the cursor object and **returns an int** of the number of rows affected
cursor.executemany()	Executes/runs multiple instances of the given command/query with sequential data and **returns an int** of the number of rows affected (if any)
cursor.fetchone()	**Returns a dict** (if using DictCursor cursorclass) or **a tuple** (if using Cursor cursorclass) of a single row of results from the previous execute method; Cursor starts on the first row and scrolls down to the next row each time the method is called
cursor.fetchall()	**Returns a list of dicts** (if using DictCursor cursorclass) or **a tuple of tuples** (if using Cursor cursorclass) of all the rows of results from the previous execute method.
connection.commit()	**Returns None** and and ends the current transaction with the database and saves all changes made
connection.close() / cursor.close()	**Returns None** and terminates the connection/cursor with the database/connection object

Steps to Connecting to your Database

1. **Establish a connection**
    - pymysql.connect()
2. **Create a Cursor Object**
    - connection.cursor()
3. **Execute your SQL Query/Statements**
    - cursor.execute() / cursor.executemany()
4. **Close your cursor after executing all statements**
    - cursor.close()
5. **Commit your connection**
    - connection.commit()
6. **Close the connection**
    - connection.close()

Running Our Example Code

Using methods discussed above, we have created an example Python file that uses PyMySQL to modify and print results of an existing database named `CS2316`. Feel free to trace and modify your downloaded copies of the .py files.

Running pymysql_dictcursor.py

Before running our Python file, first run the `recitation_tas.sql` schema script. The following is our code from a Python file named `pymysql_dictcursor.py`, which uses the **DictCursor cursorclass**:

```
import sys, pymysql
from pprint import pprint

def main():
    connection = pymysql.connect(host = 'localhost', user = 'root', password = sys.argv[1], db = sys.argv[2],
                     charset = 'utf8mb4', cursorclass = pymysql.cursors.DictCursor) # creates connection object

    cursor = connection.cursor() # creates a cursor object
    query = 'INSERT INTO roster VALUES (0, "Melinda McDaniel", "CS", 0);'
    cursor.execute(query) # executes the INSERT query

    num_of_rows_affected = cursor.execute('SELECT * FROM recitation;') # saves the # of rows affected from query as an int
    print(f'Number of rows affected: {num_of_rows_affected}\n')

    cursor.execute('SELECT * FROM recitation;') # selects all rows from recitation table
    recitation_dict = cursor.fetchone() # returns the 1st row from the previous query
    print('This is the first row of the previous query!')
    pprint(recitation_dict)
    cursor.close() # terminates the cursor object
    # other method of creating a cursor object

    with connection.cursor() as cursor: # creates temporary cursor that closes once it reaches out of the indentation
        data = [(8, "Dwight Schrute", "Marketing", 0), (9, "Jim Halpert", "Marketing", 0)]
        cursor.executemany('INSERT INTO roster VALUES (%s, %s, %s, %s);', data) # inserts all info from data

        cursor.execute('SELECT * FROM roster;') # selects all rows from roster table
        roster_list = cursor.fetchall() # returns all row data from previous query
        print('\nThese are all the rows of the previous query!')
        pprint(roster_list)

    connection.commit()
    connection.close()

if __name__ == '__main__':
    main()
```

We have created some query statements that modify the `roster` table by inserting new data. We have also included printing both the `.fetchone()` and `.fetchall()` methods to show how they differ from each other. Since we set `password = sys.argv[1]` and `db = sys.argv[2]`, we will pass in variables when running our script with Terminal / Command Prompt. Below is what is printed after running the `pymysql_dictcursor.py` file as a script:

Terminal/Command Prompt:

```
(base) bohongcheng@host-68-234-129-225 Desktop % python pymysql_dictcursor.py MySQLPassword CS2316
Number of rows affected: 4

This is the first row of the previous query!
{'num_students': 0, 'room': 'CoC 273B', 'section': 'Helpdesk', 'ta_ID': 0}

These are all the rows of the previous query!
[{'ID_number': 0, 'major': 'CS', 'name': 'Melinda McDaniel', 'ta_ID': 0},
 {'ID_number': 1, 'major': 'ISYE', 'name': 'Manny Jonson', 'ta_ID': 0},
 {'ID_number': 2, 'major': 'ISYE', 'name': 'Xinran Yu', 'ta_ID': 1},
 {'ID_number': 3, 'major': 'CS', 'name': 'Eric Yan', 'ta_ID': 3},
 {'ID_number': 4, 'major': 'ISYE', 'name': 'Colleen Cabugason', 'ta_ID': 2},
 {'ID_number': 5, 'major': 'ISYE', 'name': 'Marylyn Chen', 'ta_ID': 2},
 {'ID_number': 6, 'major': 'ISYE', 'name': 'Josh Hall', 'ta_ID': 1},
 {'ID_number': 7, 'major': 'ISYE', 'name': 'Bohong Cheng', 'ta_ID': 3},
 {'ID_number': 8, 'major': 'Marketing', 'name': 'Dwight Schrute', 'ta_ID': 0},
 {'ID_number': 9, 'major': 'Marketing', 'name': 'Jim Halpert', 'ta_ID': 0}]
```

Here is our modified `roster` table after running our .py file:

```
+-----------+-------------------+-----------+-------+
| ID_number | name              | major     | ta_ID |
+-----------+-------------------+-----------+-------+
|         0 | Melinda McDaniel  | CS        |     0 |
|         1 | Manny Jonson      | ISYE      |     0 |
|         2 | Xinran Yu         | ISYE      |     1 |
|         3 | Eric Yan          | CS        |     3 |
|         4 | Colleen Cabugason | ISYE      |     2 |
|         5 | Marylyn Chen      | ISYE      |     2 |
|         6 | Josh Hall         | ISYE      |     1 |
|         7 | Bohong Cheng      | ISYE      |     3 |
|         8 | Dwight Schrute    | Marketing |     0 |
|         9 | Jim Halpert       | Marketing |     0 |
+-----------+-------------------+-----------+-------+
10 rows in set (0.00 sec)
```

Running pymysql_cursor.py

Before running our Python file, first run the `recitation_tas.sql` schema script. The following is our code from a Python file named `pymysql_cursor.py`, which uses the **Cursor cursorclass**:

```
import sys, pymysql
from pprint import pprint

def main():
    connection = pymysql.connect(host = 'localhost', user = 'root', password = sys.argv[1], db = sys.argv[2],
                    charset = 'utf8mb4', cursorclass = pymysql.cursors.Cursor) # creates connection object

    cursor = connection.cursor() # creates a cursor object
    query = 'INSERT INTO roster VALUES (0, "Melinda McDaniel", "CS", 0);'
    cursor.execute(query) # executes the INSERT query

    num_of_rows_affected = cursor.execute('SELECT * FROM recitation;') # saves the # of rows affected from query as an int
    print(f'Number of rows affected: {num_of_rows_affected}\n')

    cursor.execute('SELECT * FROM recitation;') # selects all rows from recitation table
    recitation_tuple = cursor.fetchone() # returns the 1st row from the previous query
    print('This is the first row of the previous query!')
    pprint(recitation_tuple)
    cursor.close() # terminates the cursor object
    # other method of creating a cursor object

    with connection.cursor() as cursor: # creates temporary cursor that closes once it reaches out of the indentation
        data = [(8, "Dwight Schrute", "Marketing", 0), (9, "Jim Halpert", "Marketing", 0)]
        cursor.executemany('INSERT INTO roster VALUES (%s, %s, %s, %s);', data) # inserts all info from data

        cursor.execute('SELECT * FROM roster;') # selects all rows from roster table
        roster_tuple = cursor.fetchall() # returns all row data from previous query
        print('\nThese are all the rows of the previous query!')
        pprint(roster_tuple)

    connection.commit()
    connection.close()

if __name__ == '__main__':
    main()
```

We have created some query statements that modify the `roster` table by inserting new data. We have also included printing both the `.fetchone()` and `.fetchall()` methods to show how they differ from each other. Since we set `password = sys.argv[1]` and `db = sys.argv[2]`, we will pass in variables when running our script with Terminal / Command Prompt. Below is what is printed after running the `pymysql_dictcursor.py` file as a script:

Terminal/Command Prompt:

```
(base) bohongcheng@host-68-234-129-225 Desktop % python pymysql_cursor.py MySQLPassword CS2316
Number of rows affected: 4

This is the first row of the previous query!
(0,'Helpdesk', ‘CoC 273B’)

These are all the rows of the previous query!
((0, 'CS', 'Melinda McDaniel', 0),
 (1, 'ISYE', 'Manny Jonson', 0),
 (2, 'ISYE', 'Xinran Yu', 1),
 (3, 'CS', 'Eric Yan', 3),
 (4, 'ISYE', 'Colleen Cabugason', 2),
 (5, 'ISYE', 'Marylyn Chen', 2),
 (6, 'ISYE', 'Josh Hall', 1),
 (7, 'ISYE', 'Bohong Cheng', 3),
 (8, 'Marketing', 'Dwight Schrute', 0),
 (9, 'Marketing', 'Jim Halpert', 0))
```

Here is our modified `roster` table after running our .py file:

```
+-----------+-------------------+-----------+-------+
| ID_number | name              | major     | ta_ID |
+-----------+-------------------+-----------+-------+
|         0 | Melinda McDaniel  | CS        |     0 |
|         1 | Manny Jonson      | ISYE      |     0 |
|         2 | Xinran Yu         | ISYE      |     1 |
|         3 | Eric Yan          | CS        |     3 |
|         4 | Colleen Cabugason | ISYE      |     2 |
|         5 | Marylyn Chen      | ISYE      |     2 |
|         6 | Josh Hall         | ISYE      |     1 |
|         7 | Bohong Cheng      | ISYE      |     3 |
|         8 | Dwight Schrute    | Marketing |     0 |
|         9 | Jim Halpert       | Marketing |     0 |
+-----------+-------------------+-----------+-------+
10 rows in set (0.00 sec)
```

PyMySQL Alternatives

Just like with almost every software available, there exists alternative programs that can achieve the same results that you want. Some notable alternatives to PyMySQL are MySQL Connector-Python, mySQLdb, and mysqlclient. Each of these alternatives provide the same functionality as PyMySQL, but are structured differently. Although these are all database connectors for Python to MySQL, each has their separate pros and cons. In this course, **we will primarily work with PyMySQL**, so knowledge on these alternative database connectors **will not be tested**.

Although all of these are similar, PyMySQL requires CPython to function and is more popular than mySQLdb, whereas mysqlclient is essentially a forked version of mySQLdb. For more information on these alternatives, feel free to Google them!

References

Here are a couple of links to documentation, code structure examples, helpful resources, and alternative SQL/Python connectors that were not discussed:

PyMySQL Documentation: https://pymysql.readthedocs.io/en/latest/

PyMySQL tutorial:
https://o7planning.org/en/11463/connecting-mysql-database-in-python-using-pymysql

Additional PyMySQL tutorial: http://zetcode.com/python/pymysql/

PyMySQL example: https://pymysql.readthedocs.io/en/latest/user/examples.html

Alternative SQL/Python connectors:

- **MySQL Connector-Python Documentation:** https://dev.mysql.com/doc/connector-python/en/
- **MySQLdb Python Documentation:** https://mysqlclient.readthedocs.io/user_guide.html#introduction
- **mysqlclient Documentation:** https://pypi.org/project/mysqlclient/